TEAM HANDOFF • FINALIZED PROJECT SCOPE

Railway Reservation System

PROJECT BRIEF | Simplified IRCTC-style DBMS website

Purpose. Share the agreed website features, DBMS requirements, deployment choice and a complete Antigravity build prompt with the team. The formal DBMS report is a separate workstream and is deliberately excluded from the Antigravity prompt.

At a glance

Area	Final choice
Development	Google Antigravity
Frontend	HTML, CSS, vanilla JavaScript + Jinja2
Backend	Python Flask; simple forms/routes; no separate REST API
Database	PostgreSQL + psycopg (direct, parameterized SQL)
Repository / public deployment	GitHub + Render Flask web service + Render PostgreSQL
ER diagram	Editable .drawio file plus PNG/SVG export
Formal report / DBMS concepts write-up	Prepared independently; NOT generated by Antigravity

Scope guardrails: No React, microservices, ORM, real payments, live IRCTC integration, RAC/waitlist, Tatkal, physical berth assignment or complex fare engines. No member allocations, PPT plan or date-based schedule in this brief.

01 Agreed website features

These are the intended passenger-facing requirements. They should work with real PostgreSQL records, not static or hardcoded pages.

Module	Required behavior
Accounts	Registration, login/logout, hashed passwords, simple sessions and access control.
Search	Source, destination and journey date; train number/name, correct station ordering, timetable and intermediate stops.
Class / availability	Select SL, 3AC, 2AC etc.; view class-wise capacity for each train run/date.
Fare	Display fare and compute final total on the server.
Booking	Book multiple passengers under one PNR; commit the booking atomically and reject over-capacity requests.
Status / history	Limited PNR status lookup, signed-in booking history and full ticket details for the account owner.
Cancellation	Cancel own booking; keep history with CANCELLED status and release reserved capacity.
E-ticket	Printable booking confirmation/ticket using HTML and print CSS.

Optional, after all core flows work

- Small admin panel to add/edit trains and stations/routes; basic booking, cancellation and revenue summaries.

Agreed simplifications

- No assigned seat or berth number. A passenger uses one capacity unit of the selected train run and class.
- For simplicity, capacity is reserved for the entire train run, even if travel covers only part of the route. Route order is still validated.
- Train runs are date-specific. No real railway timetable or payment provider is needed: sample data is fictional.

- Searching and fare previews can use JavaScript, but Flask/PostgreSQL decide availability, payment-free booking and cancellation.

02 DBMS evaluation requirements

The database is the main academic deliverable: Antigravity should generate artifacts that students can open, inspect and explain.

Concept	Expected evidence
Conceptual model	ER diagram with entities, attributes, PKs, relationships, cardinalities and participation as relevant.
Relational design	Relational schema, PK/FK/candidate keys and valid referential integrity constraints.
Functional dependencies	Important FDs and candidate keys for every relation; BCNF where achievable, otherwise justified 3NF.
SQL	DDL/DML, JOINs, meaningful subquery, GROUP BY/HAVING, COUNT/SUM/AVG and ORDER BY.
Constraints	NOT NULL, UNIQUE, CHECK, DEFAULT, foreign keys and relevant business rules.
Transactions	COMMIT/ROLLBACK and concurrency-safe booking/cancellation preventing overbooking.

03 Database structure and mandatory files

Antigravity should propose roughly 8–10 tables; the final count depends on normalization and actual relationships. Suggested starting entities:

Relation	Purpose
Users	Registered accounts; password hashes and role if admin exists.
Stations	Station codes and names.
Trains	Unique train numbers and names.

Relation	Purpose
Train_Stops	Ordered station stops, arrival/departure information per train.
Train_Runs	Specific train service on a calendar date.
Travel_Classes	SL, 3AC, 2AC etc.
Train_Class_Capacity	Capacity and fare for a run and class (or a properly designed equivalent).
Bookings	PNR, owner, train run, boarding/destination and status.
Booking_Passengers	Passenger details linked to a booking and selected class.

Antigravity-generated project assets

- docs/Railway_Reservation_ER_Diagram.drawio - a REAL editable diagram, not only a screenshot or Mermaid text.
- docs/ER_Diagram.png (or SVG) - image export of that same ER design.
- sql/schema.sql, sql/seed.sql, sql/demo_queries.sql - tables, fictional data and demonstrable SQL.
- app.py, templates/, static/css/, static/js/ - Flask and website code.
- requirements.txt, .env.example, .gitignore, README.md - dependencies, example configuration and setup guide.

> Important: The Antigravity docs/ folder is for the ER diagram and supporting technical assets. Do not ask it for DBMS_Project_Report_Draft.md or DBMS_Concepts_Implemented.md. Those two deliverables belong to the separate reporting process.

04 Deployment and tests

Chosen simple deployment: push the Flask project to GitHub, connect a Render web service, and use Render PostgreSQL with the database URL stored as an environment variable. Antigravity can prepare Gunicorn configuration and instructions.

Current Render free-tier caveat: a free web service spins down after 15 minutes idle and may need about a minute to restart; a free PostgreSQL database expires 30 days after creation. Keep local SQL/seed files so the database can be reconstructed. Source: render.com/docs/free.

Must-test flows

- Register/login; correct route direction; search by date; availability by class; fare calculation.
- Book multiple passengers; unique PNR; retrieve own booking; print; cancel and recheck availability.
- Reject bookings beyond capacity; rollback on errors; avoid concurrent overbooking.
- Block unauthorized access; keep secrets out of Git; validate the public deployed version.

05 Formal report: separate from Antigravity

> Separate workstream: Prepare the final report using another tool or manual writing. Antigravity should not generate report Markdown files. You can use its final code, SQL scripts, ER diagram and real test evidence as source material for the report.

Recommended report contents

Section	What to include
1. Abstract / introduction	Project purpose, users and reason for a DBMS-backed reservation system.
2. Problem statement / objectives	What the application supports and which limitations are deliberate.
3. Requirements	Functional and non-functional requirements; assumptions and scope exclusions.
4. Architecture	Browser -> Flask/Jinja2 -> psycopg -> PostgreSQL; local vs public setup.
5. ER diagram	The actual editable ER diagram, entities, relationships and cardinalities.
6. Relational schema	Every table, primary/candidate/foreign keys, relationships and constraints.
7. FDs and normalization	1NF/2NF/3NF verification; BCNF analysis or justified 3NF exceptions.
8. SQL implementation	Selected CREATE TABLE, JOIN, subquery, aggregates, GROUP BY/HAVING and DML examples.
9. Transactions / integrity	Availability rules, booking/cancellation transaction, rollback and overbooking prevention.
10. Integration	How Flask calls SQL through psycopg and renders the website.
11. Testing / screenshots	Actual test cases with inputs, expected and observed outputs; screenshots.
12. Limitations / future scope	No berth assignment, capacity per full run, no live data/payments; possible extensions.
13. Conclusion	What was built and which DBMS concepts it demonstrates.

DBMS concepts evidence sheet (also separate)

Keep a compact concept-to-proof checklist while reviewing implementation. This can live in your own notes, report appendix or separate document; it must not be an Antigravity-generated .md file.

Concept	Evidence to collect
Normalization	FDs, candidate keys, BCNF/3NF reasoning and final relation definitions.
SQL / constraints	SQL snippets, table definitions and successful/failed constraint examples.
Joins / aggregates	Queries plus screenshots or result sets from real PostgreSQL data.
Transactions	Booking SQL code, rollback test and a concurrent-booking result.
Application integration	Flask route + query + resulting web page screenshot.

06 Build sequence (no dates, no role assignments)

The following is the logical work order, not a schedule:

- Agree requirements and documented assumptions.
- Review ER model, relationships and editable draw.io file.
- Approve relational schema, FDs and BCNF/3NF analysis.
- Create PostgreSQL schema, seed data and demonstration SQL.
- Implement Flask routes, bookings, availability and cancellation.
- Add HTML/CSS/JavaScript UI, test with real PostgreSQL records.
- Push to GitHub, configure Render and verify public access.

Acceptance checklist before sharing the finished project

- Every core feature works in the browser and updates/reads PostgreSQL correctly.
- ER .drawio is editable and matches schema.sql.
- Important SQL statements and normal forms can be explained in a viva.
- No credentials appear in the repository; tests and demo data are reproducible.
- Report is authored separately using verified implementation evidence.

> Next: Add the three professor PPTs to Antigravity and confirm they can be read.
> Then paste the updated master prompt in Part II (also supplied as a text file).

07 BEFORE PASTING THE MASTER PROMPT

Prepare the three reference PPTs

Complete these steps in **Google Antigravity before running the master prompt**. This ensures the database follows your lecturer's ER-modeling and SQL conventions.

Step 1 - Open the project workspace

Open Antigravity and create or open the **Railway Reservation System** workspace. Keep the database, application code and reference materials in the same project.

Step 2 - Add the three professor-provided PPTs

Create a folder called `references/` and put **all three files** inside it:

`S_ER Model.pptx`
: ER notation, attributes, keys, relationships, cardinality and participation constraints.

`Conversion of ERD to Relational Model_1(1).pptx`
: Your lecturer's seven ER-to-relational mapping steps.

`SQL_Chapter_DB_1(2).pptx`
: SQL DDL, DML, constraints, joins, aggregates, subqueries, views and triggers.

Step 3 - Make sure Antigravity can read the PPTs

Ask Antigravity to confirm it can read each PPT. If a file is unreadable, export it as PDF and add that version to `references/`. Never claim to have followed unreadable slides.

Step 4 - Paste the master prompt from the next section

After the PPTs are accessible, paste the complete master prompt. Include this instruction:

"Read all three reference PPTs before designing the database. Follow the lecturer's ER conventions, seven ER-to-relational mapping rules and SQL topics. Explain which mapping rules you apply."

Step 5 - Review the design before generating the website

Review the **editable draw.io ER diagram, step-by-step relational mapping and BCNF/3NF analysis** first. Correct the design before Antigravity generates the database and website.

Reminder: The formal DBMS report and concepts evidence sheet are **separate work**, as explained earlier. Do not ask Antigravity to generate these, a PPT, team roles or a schedule.

PART II / READY-TO-PASTE ANTIGRAVITY PROMPT

Build instructions

The prompt below is the updated version. Use it **after placing the three professor PPTs in the Antigravity workspace**. It excludes the formal report, concepts write-up, PPT, timelines and member responsibilities.

ANTIGRAVITY MASTER PROMPT - RAILWAY RESERVATION SYSTEM

Updated for professor's ER-modeling, ER-to-relational mapping, and SQL PPTs

1. Project goal

Build a complete but simple Railway Reservation System for a second-year B.Tech DBMS course project. It should resemble a simplified IRCTC-style website but use fictional demonstration data. Emphasize correct ER modeling, conversion to relations, normalization, SQL queries, integrity constraints, transactions and a working end-to-end application.

Keep the implementation easy for students to understand and explain in a DBMS viva. Avoid unnecessary frameworks, complexity, advanced features and separate API services. Do not add unrequested features.

2. Required technology stack

- Development environment: Google Antigravity
- Frontend: HTML, CSS, vanilla JavaScript and Jinja2 templates
- Backend: Python Flask, with server-rendered routes and HTML forms
- Database: PostgreSQL
- Python database connection: psycopg, executing direct parameterized SQL (NO ORM)
- Version control: Git / GitHub
- Public deployment: Render for Flask and, if available on a suitable free plan, Render PostgreSQL
- ER diagram: editable diagrams.net / draw.io file (.drawio) and an image export

Do NOT introduce React, Next.js, FastAPI, separately hosted REST APIs, SQLAlchemy, Docker, microservices, live IRCTC integration, real payment gateways or other paid/unnecessary dependencies. JavaScript supports interactivity; Flask/PostgreSQL remain the source of truth for bookings, fares and availability.

3. Reference PPTs - IMPORTANT

I will also provide these three professor-supplied PPT files:

1. S_ER Model.pptx
2. Conversion of ERD to Relational Model_1(1).pptx
3. SQL_Chapter_DB_1(2).pptx

Read all three if they are accessible in the Antigravity workspace. Treat the first two as the reference for ER modeling and ER-to-relational conversion, and the third as the reference for our SQL topics and course-level emphasis.

If you cannot access any of these PPTs, explicitly say which ones you could not read. Do not claim to have followed unseen slides. Ask for an accessible exported PDF/text version if needed. Do not replace our lecturer's conventions with invented rules.

4. Website features (core requirements)

A. Users

- Register, log in and log out using securely hashed passwords and simple sessions.
- Require login to make bookings, view full booking history or cancel one's own bookings.

B. Train search and information

- Search trains by source station, destination station and journey date.
- Display train numbers/names, ordered intermediate stations, departure and arrival schedules, and journey duration.
- Support travel classes such as SL, 3AC and 2AC.
- Show date-wise, class-wise seat availability and fares.

C. Booking

- Book multiple passengers under a single booking and unique PNR.
- Validate passenger information and capacity in the backend.

- Save booking and passenger rows together in an atomic PostgreSQL transaction.
- Prevent overbooking even when two requests try to book the last places concurrently.
- Compute actual fare on the backend; JavaScript may only display a preview.

D. Manage bookings

- Enquire about limited PNR/booking status without exposing personal passenger data publicly.
- Show a logged-in user's booking history and full ticket details.
- Cancel an authorized booking; retain the record with CANCELLED status and restore capacity.
- Display a printable e-ticket using HTML and print-friendly CSS (no generated PDF necessary).

Optional after the core is working: a simple admin panel to manage trains/stations/routes and show booking, cancellation and fare summaries. Restrict admin access.

5. ER diagram - follow S_ER Model.pptx

Design an ER conceptual model FIRST, rather than making SQL tables and drawing a diagram afterward.

Use the ER ideas and conventions taught in the professor's S_ER Model.pptx, including as applicable:

- Strong/regular entities, weak entities only if genuinely justified
- Attributes: simple/composite, multivalued and derived, when appropriate
- Entity identifiers / key attributes
- Relationships, their degree, and relationship attributes
- Cardinality ratios (1:1, 1:N, M:N)
- Total/partial participation and minimum-maximum constraints where useful
- Proper distinction between an entity, an attribute and a relationship

Do not invent weak entities or multivalued attributes solely to showcase a rule. Explain the meaning and participation of each actual relationship.

Generate a real editable draw.io XML file at:

docs/Railway_Reservation_ER_Diagram.drawio

Also export an image as:

docs/ER_Diagram.png (or docs/ER_Diagram.svg)

The .drawio file must open and be editable at https://app.diagrams.net. Do not substitute Mermaid, a picture only or a text description. Verify that the ER diagram accurately represents the chosen design.

6. Explicit ER-to-relational conversion - follow conversion PPT

Show the mapping from the ER model to relational schemas BEFORE writing CREATE TABLE statements. Follow the seven ER-to-relational mapping steps taught in Conversion of ERD to Relational Model_1(1).pptx, whenever the corresponding construct occurs:

1. Regular entity: create a relation with its simple attributes and primary key.
2. Weak entity: create its relation, including the owner's key as a foreign key and the weak entity's partial key as appropriate.
3. Binary 1:1 relationship: use the appropriate foreign key approach, preferably on the total-participation side; consider the PPT's alternatives where justified.
4. Binary 1:N relationship: place the one-side's key as a foreign key on the many-side; account for relationship attributes.
5. Binary M:N relationship: create a separate relationship relation containing the participating keys and relationship attributes.
6. Multivalued attribute: create a separate relation containing the owner key and the attribute's value/components.
7. N-ary relationship: create a relationship relation with participating entity keys and relevant relationship attributes.

Only apply steps for constructs that actually exist in the ER diagram. Explicitly identify which mapping rule produced each resulting relation. Show primary keys, candidate keys, foreign keys, composite keys where needed and referential integrity. Make the final relational schema, ER diagram and actual SQL tables mutually consistent.

7. Normalization

For each relation, identify relevant functional dependencies and candidate keys. Verify 1NF, 2NF and 3NF, then aim for BCNF wherever feasible. If a relation stays in 3NF, provide a real, specific justification (such as preserving dependencies that BCNF decomposition would lose). Do not blindly claim that all relations are BCNF.

Aim for approximately 8-10 relations, but add or remove tables as demanded by correct modeling; do not sacrifice normalization to meet an arbitrary number.

Likely entities/relations to consider: Users, Stations, Trains, Train_Stops, Train_Runs, Travel_Classes, Train_Class_Capacity, Bookings and Booking_Passengers. Finalize them based on the actual ER model and functional dependencies.

8. SQL - follow SQL_Chapter_DB_1(2).pptx

This is the main DBMS implementation. Use appropriate SQL constructs from the professor's SQL chapter, not a complicated database abstraction.

MANDATORY - demonstrate with real schema/data and executable queries:

- DDL: CREATE TABLE and suitable PostgreSQL data types; show ALTER TABLE where meaningful.
- Constraints: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, DEFAULT, and suitable referential actions.
- DML: INSERT, SELECT, UPDATE, and DELETE where appropriate (do not delete booking history when cancelling).
- Queries: SELECT/FROM/WHERE, DISTINCT, ORDER BY, aliases, comparison and filtering.
- Joins: INNER JOIN and LEFT JOIN when relevant; join train/station/route/booking tables.
- Aggregates: COUNT, SUM, AVG, GROUP BY and HAVING with practical booking/occupancy examples.
- Subqueries: at least one useful nested subquery and one meaningful correlated subquery using EXISTS or NOT EXISTS, where appropriate.
- VIEW: create at least ONE real, meaningful SQL VIEW (for example, a train booking summary or confirmed-booking overview) and demonstrate querying it. Views are mandatory for this project.

- Transactions: booking and cancellation must be atomic; commit successful changes and roll back on failure. Use suitable row locking or an equivalent concurrency-safe design to prevent overbooking.
- Use direct SQL with psycopg placeholders/parameters, never construct SQL by concatenating user input.

OPTIONAL - only if they are simple and genuinely helpful:

- One window-function query, e.g. RANK() / ROW_NUMBER() for an optional train popularity or booking report. Window functions are NOT required for the core website and are not a main topic in the supplied SQL PPT.
- One small PostgreSQL trigger with an associated trigger function, e.g. a booking-cancellation audit, ONLY if it serves a real purpose and is easy to explain. The professor's PPT introduces triggers, but they are not essential to our app.
- Other small SQL features, such as an additional view or a simple index, if useful and understandable.

DO NOT implement merely to showcase syntax:

- CREATE ASSERTION (PostgreSQL does not implement SQL assertions).
- Complicated triggers, stored procedures/functions beyond an optional simple trigger function, heavy window-function logic or needless database-side automation.

Create:

sql/schema.sql - CREATE TABLE definitions, constraints, and the mandatory VIEW

sql/seed.sql - realistic fictional Indian railway data, repeatable or with clear reset instructions

sql/demo_queries.sql - clearly labeled executable examples of the SQL topics above, including JOINs, aggregates, GROUP BY/HAVING, nested/correlated subqueries and queries against the VIEW; optional window/trigger demos only if implemented

For any SQL feature not naturally called by a website page, it is fine to demonstrate it in sql/demo_queries.sql instead of artificially adding a new website feature. Keep optional scripts safe and clearly distinguish examples that modify data.

9. Simplified railway-booking assumptions

- Train defines a route; Train_Run denotes operation on a particular journey date.
- An origin/destination pair is valid only when the destination occurs AFTER the source among ordered train stops.

- Use per-train-run, per-class capacity. For simplicity, each confirmed passenger occupies one unit of capacity for the FULL train run even when travelling only part of the route. Describe this simplification in README.md.
- No physical seat numbers or berth allocation, overlapping segment reuse, waiting list, RAC, Tatkal, Premium Tatkal, VIKALP, dynamic fares, refunds, live train data or actual payments.
- Compute availability from capacity and confirmed passenger reservations.
- Protect capacity checks and inserts using a correct PostgreSQL transaction/lock strategy.
- Cancellation marks bookings as CANCELLED instead of deleting the booking.

10. Flask, frontend and security

Keep a single simple Flask web application with Jinja2 server-rendered pages and HTML forms. JavaScript may handle passenger-field addition/removal, client-side validation, fare previews and confirmation dialogs. The server must revalidate all important information.

Include pages/routes for home/search, results, train details and schedules, registration/login/logout, passenger booking, booking confirmation, PNR lookup, history, ticket view, cancellation and optional admin.

Use clear code, helpful comments, secure password hashes, sensible error messages, authenticated/authorized operations, environment variables for DATABASE_URL and SECRET_KEY, basic CSRF protection for state-changing requests, and safe database connection management. Use psycopg direct SQL (no ORM). Do not expose passwords or sensitive booking records publicly.

11. Minimal project files and technical documentation

Produce a beginner-friendly directory structure with:

- app.py or similarly simple Flask entry point
- templates/ and static/css/, static/js/
- sql/schema.sql, sql/seed.sql, sql/demo_queries.sql
- docs/Railway_Reservation_ER_Diagram.drawio and its PNG/SVG export
- requirements.txt, .env.example, .gitignore, README.md
- Only additional files truly needed to run/test/deploy the application

README.md should provide simple Windows setup and run instructions, what the files do, database initialization, test account details, modeling assumptions, database credentials via environment variables, and straightforward deployment steps.

CRITICAL EXCLUSION: Do NOT create the formal DBMS project report, report draft, DBMS_Concepts_Implemented.md, or any other report/concepts-implemented Markdown file. The team will prepare the formal report and DBMS-concept explanations separately outside Antigravity. The docs/ folder should hold only ER diagram assets, not those report files. Do NOT prepare a PPT or assign individual team responsibilities. Do not propose a timeline.

12. Deployment - keep it simple

Prepare the existing Flask application for public hosting via GitHub and Render, without changing frameworks or creating separate API services. Use Gunicorn, the necessary Python dependencies and an appropriate start command. For the database, use Render PostgreSQL only if a suitable free instance is available; verify the current availability/limitations at deployment time and flag any required alternative before changing the architecture.

Use DATABASE_URL and SECRET_KEY environment variables. Never commit secrets. Do not rely on the developer laptop remaining online. Mention any free-tier sleeping/expiry restrictions accurately in README.md.

13. Actual testing and final checks

Run and verify on real PostgreSQL data:

- Registration/login/logout and authorization
- Train search with correct route ordering and journey date
- Travel classes, fare lookup and available capacity
- Multi-passenger booking and unique PNR
- Insufficient-capacity rejection, rollback and concurrency/overbooking protection
- Booking history, PNR enquiry and printable ticket
- Cancellation and restored capacity
- Working SQL VIEW and representative joins, aggregates, subqueries
- Invalid/malicious input handling, forbidden access and database failures
- Public website availability when deployed

Do not invent passing test results. Fix actual failures, and ensure the final ER diagram, mapping explanation, relational schema, CREATE TABLE definitions and queries all agree.

14. Work order (NO dates)

1. Read the supplied professor PPTs; identify relevant modeling and SQL rules.
2. Present project requirements, ER conceptual model, mapping-rule explanations and proposed relational schema for review.
3. Show normalization, functional dependencies and candidate keys.
4. Generate and verify the editable draw.io ER diagram.
5. Implement PostgreSQL schema, mandatory VIEW, sample data and demo SQL queries.
6. Build and test the Flask + PostgreSQL functionality.
7. Build and connect the HTML/CSS/JavaScript frontend.
8. Test all functions and prepare the public Render deployment.

Stop after presenting the ER model, relational mapping and normalization for review BEFORE generating the entire application. Once approved, implement incrementally, and keep the source code, diagram and SQL consistent.